

PATUAKHALI SCIENCE AND TECHNOLOGY UNIVERSITY

Bag Shop Inventory Management System

A comprehensive Flutter + Supabase inventory management solution for bag retail businesses

COURSE

CIT 320 :: SD Project-II

YEAR

2026

SUPERVISOR

Md Mahbubur Rahman

Table of Contents

1 Abstract

2 Introduction

3 Objectives

4 Technology Stack

5 System Architecture

6 Database Schema & ER Diagram

7 Flow Charts

8 UI/UX Design & Figma Mockups

9 Application Features

10 Feasibility Analysis

11 Route Map & Screens

12 Conclusion & Future Work

1 Abstract

The **Bag Shop Inventory Management System** is a cross-platform application built with **Flutter** and **Supabase**, designed specifically for bag retail businesses in Bangladesh. It provides a comprehensive solution for managing products (bags with type-specific attributes), inventory across multiple warehouses, sales orders with split payment support, purchase orders with receiving workflows, customer and supplier management, and detailed reporting (sales, profit, stock valuation, supplier performance).

The system implements a **multi-tenant architecture** where each company (shop owner) has isolated data, with role-based access control supporting Owner, Manager, Staff, and Accountant roles. An offline-first sync service with an outbox pattern ensures resilience in areas with unreliable internet connectivity. The application generates professional PDF invoices and reports, supports barcode/QR scanning for rapid product lookup, and includes CSV bulk import/export for data migration scenarios.

This project fulfills the requirements of **CIT 320: Software Development Project-II** at Patuakhali Science and Technology University, demonstrating mastery of modern full-stack mobile development, database design, state management, and enterprise application architecture.

2 Introduction

2.1 Project Background

Bag retail shops in Bangladesh face significant operational challenges in inventory management. Many shops still use paper-based systems or basic spreadsheets, leading to stock discrepancies, over-ordering, lost sales due to stockouts, and difficulty tracking business performance. Existing commercial inventory solutions are either too expensive for small-to-medium bag retailers or not tailored to the specific needs of a bag shop (e.g., tracking bag types, materials, sizes, and barcodes).

2.2 Problem Statement

Bag shop owners need an affordable, easy-to-use system that can:

- Track inventory across multiple warehouses/showrooms
- Process sales quickly at the point of sale (POS)
- Manage purchase orders from suppliers
- Support split payments (cash, card, mobile banking)
- Generate insights through reports (profit, stock levels, supplier performance)
- Work offline and sync when connectivity is restored
- Support multiple staff members with role-based permissions

2.3 Solution Overview

The Bag Shop Inventory Management System addresses these needs with a modern, cross-platform mobile application. Built with Flutter for the frontend and Supabase (PostgreSQL) for the backend, it provides a responsive, premium UI with real-time data synchronization, comprehensive inventory tracking, a full POS workflow, and extensive reporting capabilities.

Project Objectives

Primary Objectives

- Develop a cross-platform inventory management app for Android, iOS, Web, and Desktop
- Implement real-time data sync using Supabase PostgreSQL backend
- Create a full Point-of-Sale (POS) system with split payment support
- Design a role-based access control system for multi-staff environments
- Build comprehensive reporting and analytics dashboards



Secondary Objectives

- Implement offline-first architecture with outbox sync pattern
- Provide barcode/QR scanning for rapid product lookup
- Generate professional PDF invoices and reports
- Support CSV bulk import/export for data migration
- Provide backend health monitoring tools for administrators

4

Technology Stack

Layer	Technology	Version	Purpose
Frontend	Flutter	3.x	Cross-platform UI framework
Language	Dart	3.0+	Primary programming language
Backend	Supabase	-	Backend-as-a-Service (PostgreSQL + Auth + Storage + Realtime)
Database	PostgreSQL	-	Relational database (via Supabase)
State Mgmt	Riverpod	2.4.9	State management and dependency injection
Navigation	Go Router	12.1.1	Declarative routing with auth guards
Charts	fl_chart	0.66	Line charts, bar charts for reports
PDF	pdf + printing	3.10 / 5.11	Invoice and report PDF generation
Barcode	mobile_scanner	3.4.0	Camera-based barcode scanning
Local Storage	SharedPreferences	2.2.2	Offline outbox queue persistence
Auth	Supabase Auth	-	

Layer	Technology	Version	Purpose
			Email/password authentication
Icons	Iconsax	0.0.8	Modern icon set
Fonts	Inter + Google Fonts	-	Professional typography

Architecture Pattern: The application follows a **Clean Architecture**-inspired layered approach with distinct data, domain, and presentation layers, combined with the **Repository pattern** for data access abstraction.

5 System Architecture

5.1 High-Level Architecture

```
graph TD
    subgraph "Flutter Client Application"
        UI["UI Layer  
Widgets & Screens"]
        State["State Management  
Riverpod Providers"]
        Repo["Repository Layer  
Data Access"]
        Sync["Sync Service  
Outbox Pattern"]
    end
```

```

    PDF["    PDF Services
    Invoice & Reports"]
    .
    end

subgraph "Supabase Backend"
    Auth["    Supabase Auth
    Email/Password"]
    DB["    PostgreSQL Database
    Multi-tenant Schema"]
    RT["    Realtime
    Subscriptions"]
    Storage["    Object Storage
    Product Images"]
    end

    UI → State
    State → Repo
    Repo →|Online| DB
    Repo →|Offline Queued| Sync
    Sync →|Background Sync| DB
    UI → PDF
    State → Auth
    Auth → Auth
    UI →|Scan| Barcode["    Barcode Scanner"]

```

5.2 Clean Architecture Layers

Presentation Layer

Flutter Widgets

- Screens (40+ screens)

Data Layer

Repositories

- 14 Repository classes
- SupabaseService wrapper

Domain Layer

Business Logic

- Domain entities
- Repository interfaces

- Widgets
(reusable components)
- Providers
(Riverpod)
- Router
(GoRouter)
- Theme
(Material 3)

- SyncService
(offline queue)
- Models (15+ data models)
- LocalDatabase
(SharedPreferences)

- Business calculations
- Validation rules
- Core constants

5.3 Offline-First Sync Architecture

```
sequenceDiagram
    participant App as Flutter App
    participant Sync as SyncService
    participant SP as SharedPreferences
    participant SB as Supabase

    App->>Sync: enqueueInsert(table, rows)
    Sync->>SP: save OutboxAction to JSON
    SP-->>Sync: OK

    Note over App: Network becomes available

    App->>Sync: syncOutbox()
    Sync->>SP: load pending actions
    SP-->>Sync: [OutboxAction...]

    loop for each action
        Sync->>SB: INSERT / UPDATE
        alt success
            Sync->>SP: remove action
        else duplicate
    end
```

```

Sync→SP: skip & remove
      else error
Sync-->App: return SyncResult with error
      end
      end

Sync-->App: SyncResult(processed, remaining)

```

5.4 Authentication Flow

```

graph TD
    Start(( )) -->|Open App| Login{Logged In?}
    Login -->|No| LoginScreen[Login Screen]
    Login -->|Yes| Dashboard[Dashboard]

    LoginScreen -->|Sign In| AuthCheck{Valid?}
    AuthCheck -->|Yes| Resolve[Resolve Company & Role]
    AuthCheck -->|No| LoginScreen

    Resolve -->|Owner| OwnerDash[Owner Dashboard]
    Resolve -->|Staff| StaffDash[Staff Dashboard]

    OwnerDash --> Admin[Admin Panel]
    Admin --> StaffMgmt[Staff Management]
    Admin --> BulkOps[Bulk Import/Export]
    Admin --> Health[Backend Health]
    Admin --> SyncQueue[Sync Queue]

```

Database Schema & ER Diagram

6.1 Entity Relationship Diagram

```
erDiagram
    companies ||--o{ products : "has"
    companies ||--o{ categories : "has"
    companies ||--o{ brands : "has"
    companies ||--o{ warehouses : "has"
    companies ||--o{ customers : "has"
    companies ||--o{ suppliers : "has"
    companies ||--o{ sales_orders : "has"
    companies ||--o{ purchase_orders : "has"
    companies ||--o{ staff : "has"
    companies ||--o{ stock_movements : "has"

    products ||--o{ inventory : "stocks"
    products ||--o{ sales_order_items : "sold in"
    products ||--o{ purchase_order_items : "ordered in"
    products ||--o{ stock_movements : "moved"
    products }o--|| categories : "belongs to"
    products }o--|| brands : "branded by"

    warehouses ||--o{ inventory : "contains"
    warehouses ||--o{ stock_movements : "originates from"
    warehouses ||--o{ sales_order_items : "ships from"
    warehouses ||--o{ purchase_order_items : "receives to"

    sales_orders ||--o{ sales_order_items : "contains"
    sales_orders }o--|| customers : "placed by"

    purchase_orders ||--o{ purchase_order_items : "contains"
    purchase_orders }o--|| suppliers : "from"

    staff ||--o{ sales_orders : "created by"
    staff ||--o{ purchase_orders : "created by"
```



```
varchar role
jsonb permissions
boolean is_active
timestamp joined_date
timestamp created_at
}
```

6.2 Database Schema Details

6.2.1 Companies (Multi-tenant Root)

Column	Type	Constraints	Description
id	UUID	PK	Primary identifier
name	VARCHAR	NOT NULL	Owner/company name
shop_name	VARCHAR	NOT NULL	Display shop name
phone	VARCHAR		Contact phone
email	VARCHAR		Contact email
owner_id	UUID	FK → auth.users	Company owner

6.2.2 Products

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	
company_id	UUID	FK → companies, NOT NULL	Tenant isolation

Column	Type	Constraints	Description
sku	VARCHAR	UNIQUE, NOT NULL	Stock Keeping Unit
name	VARCHAR	NOT NULL	Product display name
description	TEXT		Product description
category_id	UUID	FK → categories	Product category
brand_id	UUID	FK → brands	Product brand
bag_type	VARCHAR		Backpack, Handbag, etc.
material	VARCHAR		Leather, Canvas, etc.
color	VARCHAR		Primary color
size	VARCHAR		Small/ Medium/ Large
dimensions	VARCHAR		Physical dimensions
weight_grams	DECIMAL		Weight in grams
barcode	VARCHAR	UNIQUE	Barcode identifier
unit_cost	DECIMAL(12,2)	NOT NULL	Cost price

Column	Type	Constraints	Description
selling_price	DECIMAL(12,2)	NOT NULL	Retail price
wholesale_price	DECIMAL(12,2)		Bulk price
min_stock	INTEGER	DEFAULT 5	Minimum stock threshold
max_stock	INTEGER		Maximum stock cap
reorder_point	INTEGER		Reorder trigger level
is_active	BOOLEAN	DEFAULT true	Product active status
image_urls	JSONB		Array of image URLs

6.2.3 Inventory (Stock Tracking)

Column	Type	Constraints	Description
id	UUID	PK	
product_id	UUID	FK → products	Stocked product
warehouse_id	UUID	FK → warehouses	Warehouse location
quantity	INTEGER	DEFAULT 0	Current stock quantity
reserved_quantity	INTEGER	DEFAULT 0	Reserved for orders
available_quantity	INTEGER		Available (quantity - reserved)

Column	Type	Constraints	Description
batch_number	VARCHAR		Batch/lot identifier
condition	VARCHAR	DEFAULT 'NEW'	NEW, DAMAGED, etc.

6.2.4 Stock Movements (Audit Trail)

Column	Type	Constraints	Description
id	UUID	PK	
company_id	UUID	FK → companies	Tenant
product_id	UUID	FK → products	Product moved
warehouse_id	UUID	FK → warehouses	Warehouse location
movement_type	VARCHAR	NOT NULL	SALE, PURCHASE, ADJUSTMENT, TRANSFER, RETURN
quantity_change	INTEGER	NOT NULL	+/- quantity change
quantity_before	INTEGER		Snapshot before change
reference_type	VARCHAR		SALES_ORDER, PURCHASE_ORDER, MANUAL_ADJUSTMENT, STOCK_TAKE
reference_id	UUID		Link to source document
created_at	TIMESTAMP	DEFAULT now()	When movement occurred

6.2.5 Sales Orders

Column	Type	Constraints	Description
id	UUID	PK	
company_id	UUID	FK → companies	
invoice_number	VARCHAR	UNIQUE, GENERATED	Auto-generated invoice
status	VARCHAR	DEFAULT 'PENDING'	PENDING, CONFIRMED, CANCELLED
total_amount	DECIMAL(12,2)		Order total
payment_status	VARCHAR	DEFAULT 'PENDING'	PENDING, PAID, PARTIAL
payment_method	VARCHAR		CASH, CARD, UPI, SPLIT
payment_split	JSONB		Split payment breakdown

7

Flow

Charts

7.1 Point of Sale (POS) Flow

```
graph TD
    A[Open POS Screen] --> B[Search/Search Product]
    B --> C{Found?}
    C -->|Yes| D[Add to Cart]
    C -->|No| E[Try Barcode Scan]
    E --> C
    D --> F{More Items?}
    F -->|Yes| B
    F -->|No| G[Review Cart]
    G --> H[Select Payment Method]
    H --> I{CASH?}
    H --> J{CARD?}
    H --> K{UPI?}
    H --> L{SPLIT?}
    I --> M[Enter Cash Amount]
    J --> N[Enter Card Amount]
    K --> O[Enter UPI Amount]
    L --> P[Enter Split Amounts]
    M & N & O & P --> Q[Show Payment Simulation]
    Q --> R{Simulation OK?}
    R -->|Yes| S[Create Sale Order]
    R -->|No, Retry| H
    S --> T[Insert Stock Movements -SALE]
    T --> U[Generate Invoice]
    U --> V[Receipt Printed]
    V --> W[Clear Cart]
    W --> X[Return to POS]
```

7.2 Purchase Order & Receive Flow

```
graph TD
    PO[Create Purchase Order] --> POItems[Add Items & Quantities]
    POItems --> POSubmit[Submit PO]
    POSubmit --> POStatus{PENDING}

    POStatus --> |Supplier Delivers| Receive[Open Receive Screen]
    Receive --> LoadItems[Load PO Items]
    LoadItems --> CheckQty[Enter Received Quantities]
    CheckQty --> Validate{Valid?}
    Validate --> |Qty ≤ Remaining| Mvmt["Insert Stock Movements +PURCHASE"]
    Validate --> |Qty > Remaining| Error[Show Error]
    Error --> CheckQty

    Mvmt --> UpdateItems[Update received_quantity]
    UpdateItems --> CheckFull{All Received?}
    CheckFull --> |Yes| UpdateStatus[Update PO Status → RECEIVED]
    CheckFull --> |No| KeepStatus[Keep PO Status → PENDING]
    UpdateStatus --> Done[Receive Complete]
    KeepStatus --> Done
```

7.3 Inventory Adjustment Flow

```
graph TD
    Adj[Open Stock Adjustment] --> SelProd[Select Product]
    SelProd --> SelWh[Select Warehouse]
    SelWh --> GetCurr[Load Current Quantity]
    GetCurr --> EnterDelta[Enter Quantity Change]
    EnterDelta --> Reason[Enter Reason]

    Reason --> Confirm{Confirm?}
    Confirm --> |Yes| Mvmt["Create Stock Movement ADJUSTMENT"]
```

```
Confirm →|No| Cancel[Cancel]
```

```
Mvmt → Success["    Stock Updated"]
```

```
Mvmt -.→|Offline| Queue["    Queued for Sync"]
```

7.4 Stock Transfer Flow

```
graph TD
  Xfer[Open Stock Transfer] → Prod[Select Product]
  Prod → From[Select Source Warehouse]
  From → To[Select Destination Warehouse]
  To → Qty[Enter Transfer Quantity]
  Qty → Check{From has enough?}
  Check →|Yes| Batch[Optional Batch Number]
  Check →|No| Error["    Insufficient Stock"]
  Error → Qty
  Batch → Confirm2{Confirm Transfer?}
  Confirm2 →|Yes| Mvmt1["Movement 1: Source -Qty"]
  Confirm2 →|Yes| Mvmt2["Movement 2: Destination +Qty"]
  Mvmt1 & Mvmt2 → Success2["    Transfer Complete"]
```

7.5 Sales Return Flow

```
graph TD
  Open[Open Sale Detail] → Return[Click Return Items]
  Return → LoadItems[Load Sale Items]
  LoadItems → SelItems[Select Items & Quantities to Return]
  SelItems → Validate{Valid Returns?}
```

```
Validate → |Qty ≤ Original| Mvmt["Create Stock Movements +RETURN"]
Validate → |Invalid| Error["Show Error"]
Error → SelItems

Mvmt → Success[" Return Complete, Stock Restored"]
```

8

UI/UX Design & Figma Mockups

8.1 Design System

The application features a premium, professional design system built with Material Design 3 principles:

Color Palette



Primary:
Midnight Blue
(#1E3A5F)

Accent: Gold
(#D4AF37)

Semantic colors:

Typography

Headings:
Poppins (Google Fonts)

Body: Inter (Google Fonts)

Bengali Support:
NotoSansBengali

15 text style variants from displayLarge

Design Elements

- Glassmorphism containers
- Gradient buttons with shadows
- Gold accent CTA buttons
- Staggered fade-in animations
- Shimmer loading skeletons

Success,
Warning,
Danger, Info

(57px) to
labelSmall (11px)

- Premium card system

8.2 Screen Mockups (Generated)

The following UI wireframes were programmatically generated using PIL (Python Imaging Library) to represent the key screens of the application. These mockups are designed in a 1080×2400 Android canvas format and can be imported into Figma for further refinement.

Login / Register Screen

The authentication entry point with email/password login, registration for new shop owners, and password recovery options. Features a logo placeholder, form inputs, and primary action buttons styled with the brand's Midnight Blue gradient.

Route: /login, /register

Dashboard

The main landing screen showing key performance indicators (total products, low stock count, today's sales, monthly revenue) in stat cards. Features a weekly sales trend chart, quick actions grid, recent sales list, and low stock alerts. Includes a welcome hero header with the company name.

Route: /

Product List & Details

Searchable product list with category filter chips. Each product card shows name, SKU, bag type badge, selling price, and profit margin. Tapping opens the product detail screen with full information, QR code display, and edit capabilities. Category and brand management are accessible from the overflow menu.

Routes: </products>, </products/:id>

Barcode Scanner

Camera-based barcode scanning using `mobile_scanner` package. When a barcode is detected, it either looks up the existing product or offers to create a new one. Supports rapid product search for POS workflows.

Route: </scanner>

Purchase Orders

Purchase order list with create PO functionality. Each PO shows PO number, supplier, expected delivery date, and status. The receive flow allows partial receiving of items with warehouse selection and batch tracking.

Routes: </purchases>, </purchases/:id/receive>

POS / Sales

Full POS screen with product search, cart management (add/remove quantity), and checkout flow. Supports split payments (cash, card, UPI) with a payment simulation step before finalizing the sale. Generates invoice on completion.

Route: </pos>

Application Features



Authentication & Authorization

- Email/password registration and login via Supabase Auth
- Multi-tenant company isolation
- Role-based access (Owner, Manager, Staff, Accountant)
- Granular permission system (`inventory.view` , `sales.pos` , etc.)
- Staff management with invite codes



Product Management

- Bag-specific attributes (type, material, color, size, dimensions)
- Category & brand management
- Barcode & QR code generation
- Image upload via Supabase Storage
- SKU auto-generation
- Warranty tracking
- Active/inactive product status



Point of Sale (POS)

- Rapid product search by name, SKU, or barcode

Inventory Management

- Multi-warehouse support (Showroom, Warehouse, etc.)
- Batch number tracking
- Stock condition tracking (NEW, DAMAGED, etc.)
- Low stock alerts with configurable thresholds
- Stock adjustment with reason logging
- Stock transfer between warehouses
- Physical stock take with counting sessions



- Barcode scanner integration
- Cart management with quantity controls
- Multiple payment modes: Cash, Card, UPI, Split
- Payment simulation before checkout
- Automatic stock deduction on sale
- Invoice PDF generation
- Sales return workflow



Purchase Management

- Purchase order creation with line items
- Supplier management
- Partial receiving of purchase orders

Reports & Analytics

- Sales report with daily totals and top products
- Stock report with low stock identification
- Profit report (revenue, COGS, gross profit, margin %)

- Idempotency protection (clientRequestId)
- Auto-status update (PENDING → RECEIVED)
- Batch number tracking on received items
- PO number generation



- Inventory valuation report
- Supplier performance report
- Interactive charts (weekly sales trend)
- PDF export for all reports



Offline Sync

- Outbox pattern for offline-first writes
- Queue management (view, retry, clear)
- Duplicate detection via business fingerprint
- Ordered sync (oldest-first)
- Sync status indicator on dashboard
- Automatic sync on connectivity restore

Admin & Debug Tools

- Staff management (add, edit roles, permissions)
- CSV bulk import/export for products, suppliers, customers
- Backend health checker (schema + RLS validation)
- Live Supabase connection test
- Sync queue viewer
- Demo catalog seeder (30+ demo products)

Feasibility Analysis

10.1 Technical Feasibility

Factor	Assessment	Rating
Cross-Platform Support	Flutter enables single codebase for Android, iOS, Web, Linux, macOS, Windows. The app uses responsive layouts with adaptive breakpoints.	Excellent
Backend Scalability	Supabase (PostgreSQL) provides automatic scaling, row-level security, and real-time subscriptions. The multi-tenant schema with company_id isolation ensures data safety.	Excellent
Offline Capability	Outbox pattern with SharedPrefs persistence provides basic offline support. Full offline with local SQLite could be added using Drift/Hive.	Good (Basic)
Real-time Sync	Supabase Realtime channels used for staff list updates. Polling used for sales orders (10-second interval). Sufficient for inventory use case.	Good
State Management	Riverpod provides type-safe, testable state management with	Excellent

Factor	Assessment	Rating
	proper dependency injection and automatic disposal.	
PDF Generation	Using dart package 'pdf' for server-side rendering and 'printing' for platform-specific print/share dialogs. Supports Bengali fonts.	Excellent

10.2 Operational Feasibility

Factor	Assessment	Rating
Ease of Use	Premium Material 3 UI with intuitive navigation, bottom nav bar, search, and quick actions. Learning curve is minimal for users familiar with mobile apps.	High
Deployment	Supabase provides managed hosting. Flutter app can be deployed via Play Store, App Store, or web. No server maintenance required.	High
Maintenance	Clean architecture with repository pattern makes code maintainable. Riverpod's modular provider system enables easy testing and modification.	High
Data Backup		Good

Factor	Assessment	Rating
	Supabase provides automatic daily backups for PostgreSQL. Point-in-time recovery available on paid plans.	

10.3 Economic Feasibility

Cost-Benefit Analysis: The project uses free tiers for both Flutter (open-source) and Supabase (Free Tier: 500 MB database, 1 GB storage, 50,000 monthly active users). The total development cost is essentially zero beyond developer time. For a bag shop, this replaces potentially expensive commercial inventory software (typically \$50-200/month) with a custom solution at no recurring software cost.

Cost Category	Estimate	Notes
Development Tools	\$0	Flutter, VS Code, Supabase (Free Tier)
Hosting (Supabase)	\$0-\$25/month	Free for small shops; Pro (\$25) for production
App Store Deployment	\$25 (Play Store, one-time)	Apple Developer Program: \$99/year
Personnel (Student Project)	\$0	Academic project
Maintenance	~5 hours/month	Code updates, data monitoring
Total Year 1	\$25-\$400	Depends on deployment scope

10.4 Schedule Feasibility

Phase	Duration	Deliverables
Phase 1: Planning & Design	Weeks 1–2	Project proposal, ERD, UI mockups, Supabase schema
Phase 2: Core Architecture	Weeks 3–4	Flutter setup, Riverpod providers, Supabase integration, Auth
Phase 3: Product & Inventory	Weeks 5–6	CRUD for products, categories, brands, inventory, warehouses
Phase 4: Sales & POS	Weeks 7–8	POS screen, cart flow, checkout, sales list, invoice PDF
Phase 5: Purchases	Week 9	Purchase orders, receive flow, supplier management
Phase 6: Reports & Analytics	Week 10	Report screens, charts, PDF export, profit/valuation reports
Phase 7: Admin & Offline	Week 11	Admin panel, staff management, sync service, bulk tools
Phase 8: Testing & Polish	Week 12	Widget tests, integration tests, UI polish, deployment

10.5 Risk Assessment



Identified Risks

- **Internet Dependency:**
Full features require internet for Supabase. Mitigated by offline sync queue.
- **Data Loss:** Without full local database, data could be lost if sync fails. Mitigated by outbox persistence in SharedPreferences.
- **Supabase API Changes:**
Supabase Flutter SDK is actively developed. Mitigated by abstracting behind repository pattern.
- **Concurrent Access:**
Multiple staff editing same data. Mitigated by Row Level Security and realtime subscriptions.

Risk Mitigations

- Outbox pattern with retry logic for offline operations
- StableFingerprint for idempotency (preventing duplicate operations)
- Backend health checker for monitoring schema integrity
- Company-level data isolation via RLS policies
- Comprehensive error handling with user-friendly messages
- Supabase's built-in point-in-time recovery


```
Admin → BulkData["/admin/data-tools"]
Admin → AuditLog["/admin/audit-log"]
```

```
Debug["/debug/*"] → BackendHealth["/debug/backend-health"]
Debug → SyncQueue["/debug/sync-queue"]
Debug → SupabaseTest["/debug/supabase"]
```

11.1 Complete Screen Inventory

Category	Screen	Route	Permission
Auth	Login	/login	Public
Auth	Register	/register	Public
Admin	Admin Dashboard	/admin	Owner/Manager
	Staff Management	/admin/staff	staff.manage
	Bulk Data Tools	/admin/data-tools	Owner/Manager
	Audit Log	/admin/audit-log	Owner/Manager
Dashboard	Dashboard	/	All
Products	Product List	/products	products.view
	Add Product	/products/add	products.edit
	Product Detail	/products/:id	products.view
	Edit Product	/products/:id/edit	products.edit
	Categories	/products/categories	products.edit

Category	Screen	Route	Permission
Inventory	Brands	/products/brands	products.edit
	Inventory List	/inventory	inventory.view
	Analytics	/inventory/analytics	inventory.view
	Low Stock	/inventory/low-stock	inventory.view
	Adjust Stock	/inventory/adjust	inventory.edit
	Transfer Stock	/inventory/transfer	inventory.edit
	Stock Movements	/inventory/movements	inventory.view
	Stock Take	/inventory/stock-take	inventory.edit
Sales	Sales List	/sales	sales.view
	Sale Detail	/sales/:id	sales.view
	Sales Return	/sales/:id/return	sales.view
Customers	Customer List	/customers	customers.view
	Customer Form	/customers/add, /customers/:id/edit	customers.edit
Purchases	Purchase List	/purchases	purchases.view
	Add Purchase	/purchases/add	purchases.edit
	Receive Purchase	/purchases/:id/receive	purchases.edit
Reports	Reports Hub	/reports	reports.view
	Sales Report	/reports/sales	reports.view
	Stock Report	/reports/stock	reports.view

Category	Screen	Route	Permission
	Profit Report	/reports/profit	reports.view
	Inventory Valuation	/reports/valuation	reports.view
	Supplier Performance	/reports/suppliers	reports.view
POS	Point of Sale	/pos	sales.pos
Scanner	Barcode Scanner	/scanner	All
Profile	User Profile	/profile	All
Debug	Backend Health	/debug/backend-health	Admin
	Sync Queue	/debug/sync-queue	Admin
	Supabase Test	/debug/supabase	Admin

11.2 Codebase Structure

```

lib/
├── app/                # App entry point
├── core/               # Shared core utilities
│   ├── constants/     # Bag types, roles, permissions, app constants
│   ├── exceptions/    # Custom exceptions, error handlers
│   ├── utils/         # Calculations, extensions, formatters, validators, etc.
│   └── widgets/       # Shared widgets (AppBody, animations, cards, etc.)
├── data/              # Data layer
│   ├── datasources/local/ # Local database abstraction
│   ├── local/         # SharedPreferences wrapper
│   ├── models/        # 15+ data models (product, inventory, sales, etc.)
│   └── repositories/  # 14 repository classes
├── domain/            # Domain layer
│   ├── entities/      # Domain entities
│   └── repositories/  # Repository interfaces

```

```

├── presentation/                                # Presentation layer
│   ├── features/                                # Feature modules
│   │   ├── admin/                              # Admin panel (dashboard, staff, bulk data, audit)
│   │   ├── auth/                              # Auth (login, register, providers)
│   │   ├── customers/                          # Customer management
│   │   ├── dashboard/                         # Dashboard with stats
│   │   ├── debug/                             # Debug tools (health, sync, supabase test)
│   │   ├── inventory/                         # Inventory management
│   │   ├── notifications/                     # In-app notifications
│   │   ├── products/                          # Product management
│   │   ├── profile/                           # User profile
│   │   ├── purchases/                         # Purchase management
│   │   ├── reports/                           # Reports and analytics
│   │   └── sales/                             # Sales and POS
│   ├── providers/                             # Global providers (theme, connectivity)
│   ├── router/                                # GoRouter configuration
│   └── theme/                                 # Material 3 theme with custom brand colors
├── services/                                   # Services layer
│   ├── supabase_service.dart                  # Supabase client wrapper
│   ├── sync_service.dart                      # Offline sync with outbox pattern
│   ├── barcode_service.dart                   # Barcode scanning
│   ├── bulk_data_service.dart                 # CSV import/export
│   ├── invoice_pdf.dart                       # Invoice PDF generation
│   ├── report_pdf.dart                        # Report PDF generation
│   ├── notification_service.dart              # In-app notifications
│   └── backend_health_service.dart             # Backend health check

```

12.1 Conclusion

The Bag Shop Inventory Management System successfully delivers a comprehensive, production-ready inventory solution for bag retail businesses. Built with Flutter and Supabase, it demonstrates:

- **40+ screens** covering the complete retail workflow
- **15+ database tables** with proper relationships and Row-Level Security
- **14 repository classes** following clean architecture principles
- **Offline-first sync** with outbox pattern for reliable operation
- **Professional PDF** invoice and report generation
- **Premium Material 3** UI with custom brand identity
- **Role-based access control** supporting multi-staff environments

12.2 Future Enhancements



Short-term (1–3 months)

- Full offline SQLite database (Drift/Hive) for complete offline operation
- Push notifications for low stock alerts and order updates
- Supplier portal for order placement and tracking

Long-term (3–12 months)

- AI-powered demand forecasting and stock recommendations
- E-commerce integration for online + offline inventory sync
- Barcode label printing (thermal printer support)

- Multi-language support (Bengali + English)

- Analytics dashboard with advanced data visualization
- Customer loyalty program and CRM features
- Multi-currency support for international suppliers

Project Status: This project has been fully implemented and tested. The application is ready for real-world deployment with a Supabase Free Tier backend. All source code is available in the project repository.

— End of Report —

Bag Shop Inventory Management System | CIT 320 :: Software
Development Project-II
Patuakhali Science and Technology University | 2026

